



# KE&M II: Developing Ontology

---

Hutchatai Chanlekha

Department of Computer Engineering, Kasetsart University



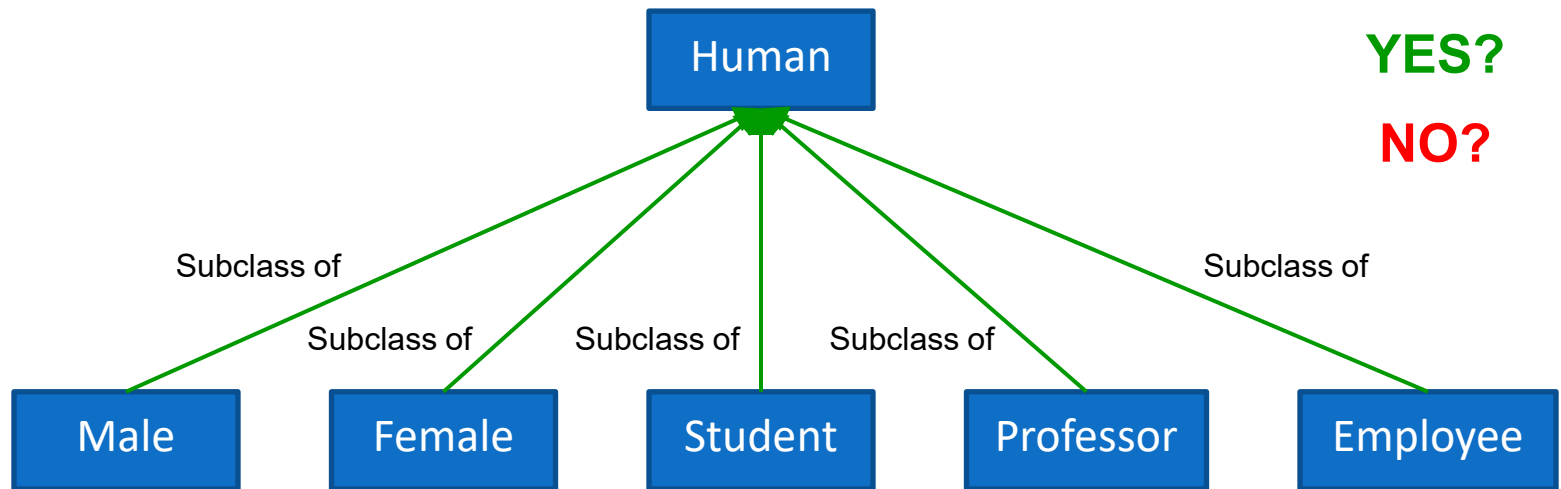


# Why performing good analysis is necessary?

- Consider a domain having several classes of people



- Should we organize these classes of people as *subclass-of* of *Human* class?





# Why performing good analysis is necessary?

- Consider a domain having several classes of people

Male

Female

Student

Professor

Employee

- Should we organize these classes of people as *subclass-of* of *Human* class?



# Let's try design hierarchy of concepts

- Suppose you want to design conceptual hierarchy of Person
- What should be subclass of People?

Person



# Fundamental rules in ontology design

- Some fundamental rules in ontology design
  - There is **no one correct way** to model a domain— there are always viable alternatives.
  - The **best solution almost always depends on the application** that you have in mind and the extensions that you anticipate.
  - Concepts in the ontology should be **close to objects (physical or logical) and relationships in your domain of interest**. These are most likely to be nouns (objects) or verbs (relationships) in sentences that describe your domain.



# A Simple Knowledge-Engineering Methodology

Most ontology development efforts have required not only the participation of ontology engineers but also of domain experts

- Ontology development is necessarily an **iterative process**.
- For iterative approach to ontology development:
  - Start with a rough first pass at the ontology.
  - Then revise and refine the evolving ontology and fill in the details.
  - Along the way, discuss the modeling decisions that a designer needs to make, as well as the pros, cons, and implications of different solutions.



# What to remember when creating ontology

- Deciding **what we are going to use the ontology for**, and **how detailed or general the ontology is going to be** will guide many of the modeling decisions down the road.
- Among several viable alternatives, we will need to determine **which one would work better for the projected task**, be more intuitive, more extensible, and more maintainable.
- Remember that an ontology is a model of reality of the world and the **concepts in the ontology must reflect this reality**.
- After we define an initial version of the ontology, we can **evaluate and debug it by using it in applications** or problem-solving methods or by **discussing it with experts** in the field, or **both**.
- This process of **iterative design** will likely continue through the entire lifecycle of the ontology.



# Step 1: Determine domain and scope of ontology

- Starting the development of an ontology by defining its domain and scope.
- That is, answer several basic questions:
  - What is the domain that the ontology will cover?
  - What are we going to use the ontology for?
  - What types of questions that the information in the ontology should provide answers for?
  - Who will use and maintain the ontology?



# Example of the design for step 1:

- Suppose we want to create ontology of wine and food.
  - Domain: Wines and food
- Purpose: support the applications that suggest good combinations of wines and food.
- Based on domain and purposes, what will be included in the ontology are:
  - Naturally, there should be concepts describing different types of wines, main food types, the notion of a good combination of wine and food
  - If the ontology will be used to help restaurant customers decide which wine to order
    - need retail-pricing information.
  - If the ontology will be used for wine buyers in stocking a wine cellar,
    - probably need wholesale pricing and availability.
  - If the people who will maintain the ontology describe the domain in a language that is different from the language of the ontology users
    - may need to provide the mapping between the languages.



# Determining scope with competency question

- One of the ways to determine the scope of the ontology is to sketch a list of questions that the ontology should be able to answer, i.e. competency questions
- These questions can be used as a test later.
  - Does the ontology contain enough information to answer these types of questions?
  - Do the answers require a particular level of detail or representation of a particular area?
- These competency questions are just a sketch and do not need to be exhaustive.



# Example of Competency questions

- Which wine characteristics should I consider when choosing a wine?
- Is *<this>* wine a *<xxx>* type of wine? [such as red or a white]
- Does *<this>* wine go well with *<this>* type of food?
- What is the best choice of wine for *<this>* dish?
- Which characteristics of a wine affect its appropriateness for a dish?
- Does a bouquet or body of *<this>* wine change with *<xxx>* vintage year?

Avoid referring to instances in the competency questions



## Step 2: Consider reusing existing ontologies

- It's worth to consider existing ontologies and checking if we can refine and extend them for our particular domain and task.
- Reusing existing ontologies may be a requirement if our system needs to interact with other applications that have already committed to particular ontologies or controlled vocabularies.
- Many ontologies are already available in electronic form and can be imported into an ontology-development environment that you are using.
- There are libraries of reusable ontologies on the Web and in the literature.
  - For example,
    - DBpedia Archivo (<https://archivo.dbpedia.org/list>)
    - Ontolingua ontology library (<http://www.ksl.stanford.edu/software/ontolingua/>)
    - DAML ontology library (<http://www.daml.org/ontologies/>).
    - There are also a number of publicly available commercial ontologies (e.g., UNSPSC ([www.unspsc.org](http://www.unspsc.org)), RosettaNet ([www.rosettanet.org](http://www.rosettanet.org)), DMOZ ([www.dmoz.org](http://www.dmoz.org))).



## Step 3: Enumerate important terms in the ontology

- It is useful to write down a list of all important terms
  - What are the terms we would like to talk about?
  - What properties do those terms have?
  - What would we like to say about those terms?

For example, important wine-related terms will include:

subtypes of wines, such as white wine, red wine, ...

different types of food, such as fish and red meat, ...

wine

wine's sugar  
content

grape

wine's color

winery

wine's body

location

wine's flavor

- Initially, it is important to get a comprehensive list of terms without worrying about overlap between concepts they represent, relations among the terms, or any properties that the concepts may have, or whether the concepts are classes or properties.



## Step 4: Define the classes and the class hierarchy

- There are several possible approaches in developing a class hierarchy

### Bottom-Up approach

- Start from the most specific concepts, i.e. the leaves of the hierarchy
- Build a structure by generalization – grouping of these classes into more general concepts

### Top-Down approach

- Start from the most general concepts in the domain
- Build a structure by specialization of the concepts
- Top concepts can be chosen in a foundational ontology

### Middle-Out approach

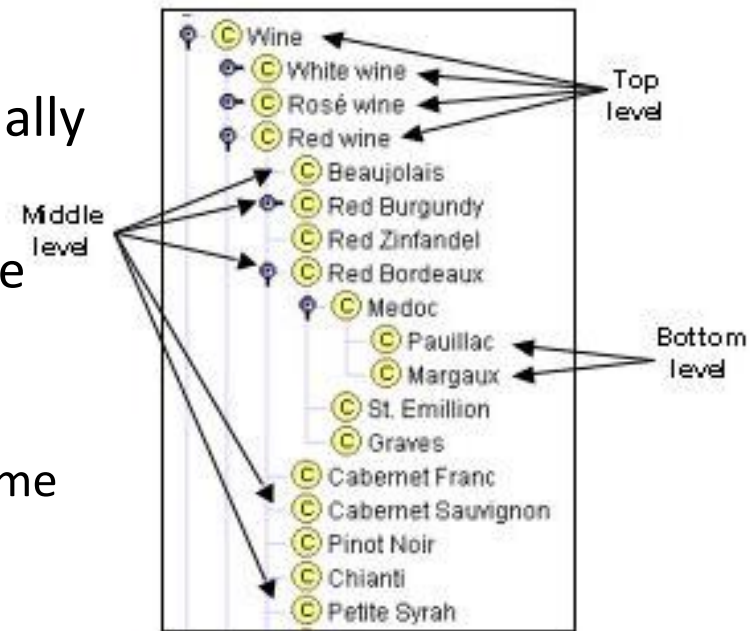
- Identify central concepts in each area/domain
- Core concepts are identified first and then generalized and specialized them appropriately to complete the ontology

None of these three methods is inherently better than any of the others. The approach to take depends strongly on the personal view of the domain.

# Organizing class hierarchy

Whichever approach we choose, we usually start by defining classes.

- From the list created in Step 3, select the terms that describe objects having independent existence
  - These terms will be classes and will become anchors in the class hierarchy.
- Organizing the classes into a hierarchical taxonomy by asking if by being an instance of one class, the object will necessarily (i.e., by definition) be an instance of some other class.
  - If a class A is a superclass of class B, then every instance of B is also an instance of A*
  - In other words, the class B represents a concept that is a “kind of” A.





## Step 5: Define the properties of classes

- The classes alone will not provide enough information to answer the competency questions from Step 1.
- Once we have defined some of the classes, we must describe the internal structure of concepts, i.e. properties
- Example:
  - After we selected classes from the list of terms we created in Step 3,
  - Most of the remaining terms are likely to be properties of these classes, for example, a wine's color, body, flavor and sugar content and location of a winery
  - For each property in the list, determine which class that you should attach these properties to.
    - For example, the Wine class will have the following properties: color, body, flavor, and sugar. And the class Winery will have a location property.



# Organizing properties of the classes

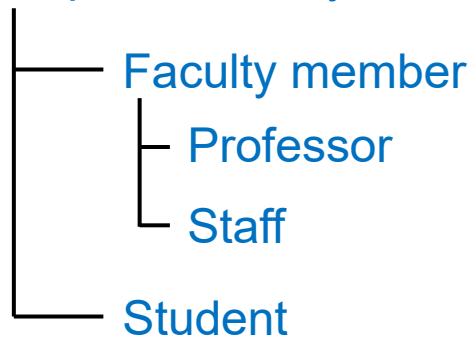
- All subclasses of a class inherit the property of that class.
  - For example, all the properties of the class Wine will be inherited to all subclasses of Wine
- A property should be attached at the most general class that can have that property.
  - For instance, body and color of a wine should be attached at the class Wine, since it is the most general class whose instances will have body and color.



# Example

Properties: name, age, citizen\_id, position, expertise, academic\_position, officeRoom, grade, academic program, student\_id

## People in Faculty





- Step 4 and 5 — developing the class hierarchy and defining properties of classes — are closely intertwined.
- Typically, we create a few definitions of the concepts in the hierarchy and then continue by describing properties of these concepts and so on.
- These two steps are also the most important steps in the ontology-design process.



## Step 6: Define the facets of the properties

- Properties can have different facets describing the value type, allowed values, the number of the values (cardinality), and other features of the values the property slot can take.
- For example,
  - The value of a “*name*” property (as in “the name of a wine”) is one string.
  - A property “*produce*” (as in “a winery produces these wines”) can have multiple values and the values are instances of the class Wine.

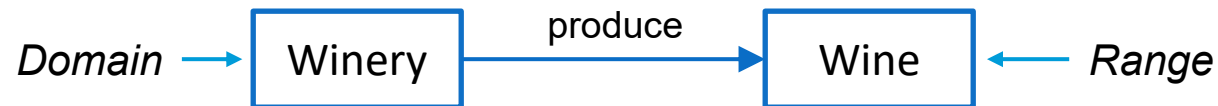


# Some common facets

- Value type
  - A value-type facet describes what types of values can fill in the property.
  - Examples of the more common value types: string, number, boolean, enumerated (specify a list of specific allowed values), other class(es)
- Cardinality
  - Cardinality defines how many values a property can have.
  - Declaration depending on the systems, for example: max, min, exact number

# Defining domain and range of a property

- The classes to which a property is attached or a classes which property describes, are called the domain.
- Allowed classes for the property value are often called a range.
- Example

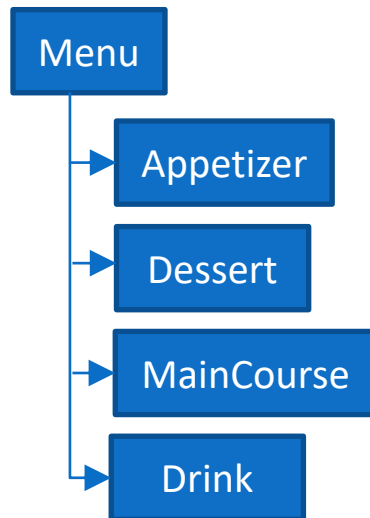


Winery must produce at least one wine

- Cardinality is “some” or “minimum 1”
- Property “*name*” from the previous example
  - Anything can have name
  - Value of “*name*” is string
    - Range of “*name*” is string
  - Everything has one name
    - Cardinality is “exactly 1”

# Defining domain and range of a property

- Basic rules for determining a domain and a range of a property
  - Find the most general classes that can be respectively the domain or the range for the property.
  - However, do not define a domain and range that is overly general
- Let's look at an example: "*hasMenu*" property



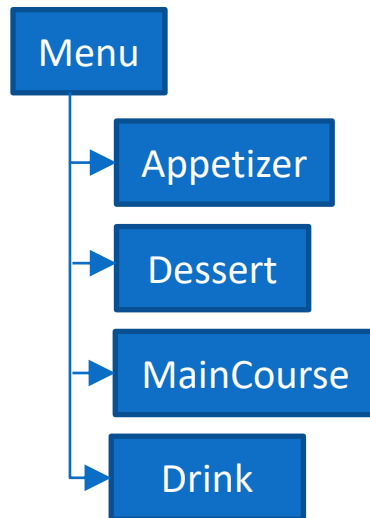
hasMenu

Domain: Restaurant

Range: Menu, Appetizer, Dessert

# Defining domain and range of a property

- Basic rules for determining a domain and a range of a property
  - Find the most general classes that can be respectively the domain or the range for the property.
  - However, do not define a domain and range that is overly general
- Let's look at an example: "*hasMenu*" property



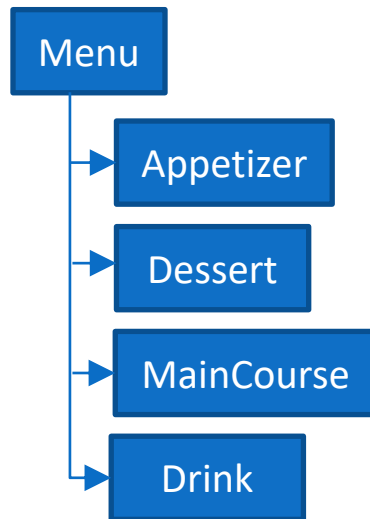
hasMenu

Domain: Restaurant

Range: Appetizer, Dessert, MainCourse, Drink

# Defining domain and range of a property

- Basic rules for determining a domain and a range of a property
  - Find the most general classes that can be respectively the domain or the range for the property.
  - However, do not define a domain and range that is overly general
- Let's look at an example: *hasMenu* property



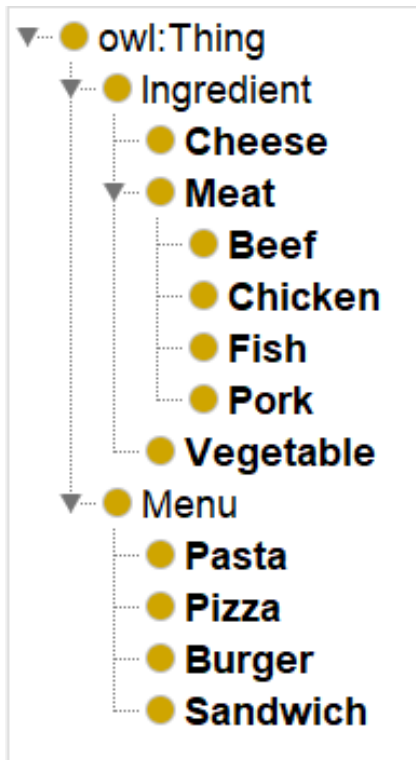
hasMenu

Domain: Restaurant

Range: Appetizer, Dessert, Drink ???

Range: Menu ???

# To define multiple domains/ranges in Protégé



- Suppose you want to define “hasFilling” property
  - Only burger and sandwich can have filling, while filling can be any ingredient
    - Domain: Burger or Sandwich
    - Range: Ingredient

Domains (intersection) -

- Burger
- Sandwich

Ranges (intersection) -

- Ingredient



Domains (intersection) +

- Burger or Sandwich

Ranges (intersection) +

- Ingredient





# Step 7: Create instances

- The last step is creating individual instances of classes in the hierarchy.
- Defining an individual instance of a class requires
  - 1) Choosing a class
  - 2) Creating an individual instance of that class
  - 3) Filling in the property values.



# Exercise: Think about database table

| Movie        | Director           | Screen Writer      | Production Company     | Release Date | Running time |
|--------------|--------------------|--------------------|------------------------|--------------|--------------|
| Spider-Man   | Sam Raimi          | David Koepp        | Columbia Pictures      | May 3, 2002  | 121          |
| The Watchers | I. Night Shyamalan | I. Night Shyamalan | Blinding Edge Pictures | June 7, 2024 | 203          |
| ...          | ...                | ...                | ...                    | ...          | ...          |

- What should be class?
- What should be property?



# Activity

- 3-4 people per group
- Create ontology for your food shop
  - Ramen shop
  - Coffee shop
  - Waffle/Pancake shop
  - Ice cream shop
  - Pasta shop
  - Sushi shop
  - Cocktail bar
  - Salad shop
  - Sandwich/Hamburger shop
  - Pastry shop
- Same topic can be chosen by not more than 2 groups



# Reference

- Slides based on
  - **Ontology Development 101: A Guide to Creating Your First Ontology**
    - Natalya F. Noy and Deborah L. McGuinness
    - [https://protege.stanford.edu/publications/ontology\\_development/ontology101.pdf](https://protege.stanford.edu/publications/ontology_development/ontology101.pdf)
    - [https://protege.stanford.edu/publications/ontology\\_development/ontology101-noy-mcguinness.html](https://protege.stanford.edu/publications/ontology_development/ontology101-noy-mcguinness.html)